



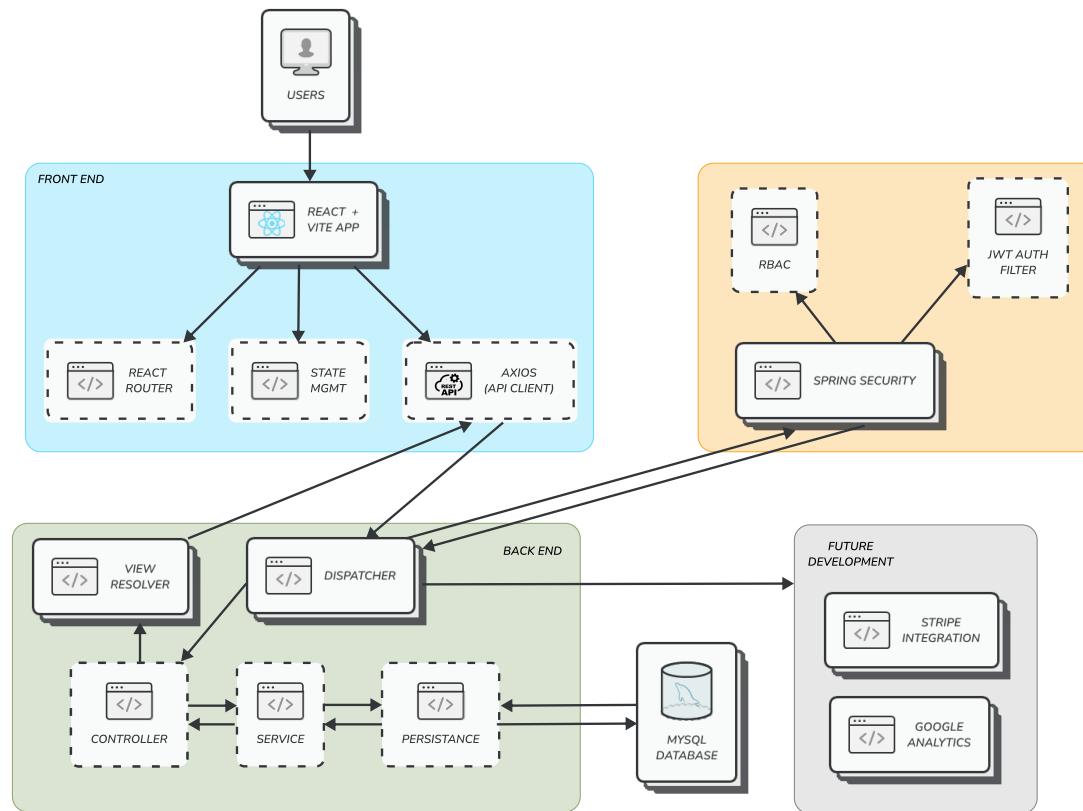
/DEV/DEPOT

By Fraser Coyle, Emily Hanson, Andrew Urso, Hoang Ho, Jordan Rodgers

Overview

- Full stack e-commerce platform for browsing and purchasing PC Components
- Built using Java Spring Boot, React.js, and MySQL.
- Backend uses RESTful API with JWT-based authentication.
- Frontend features modular components using React Context.
- Built using Agile workflows and feature driven deployment.

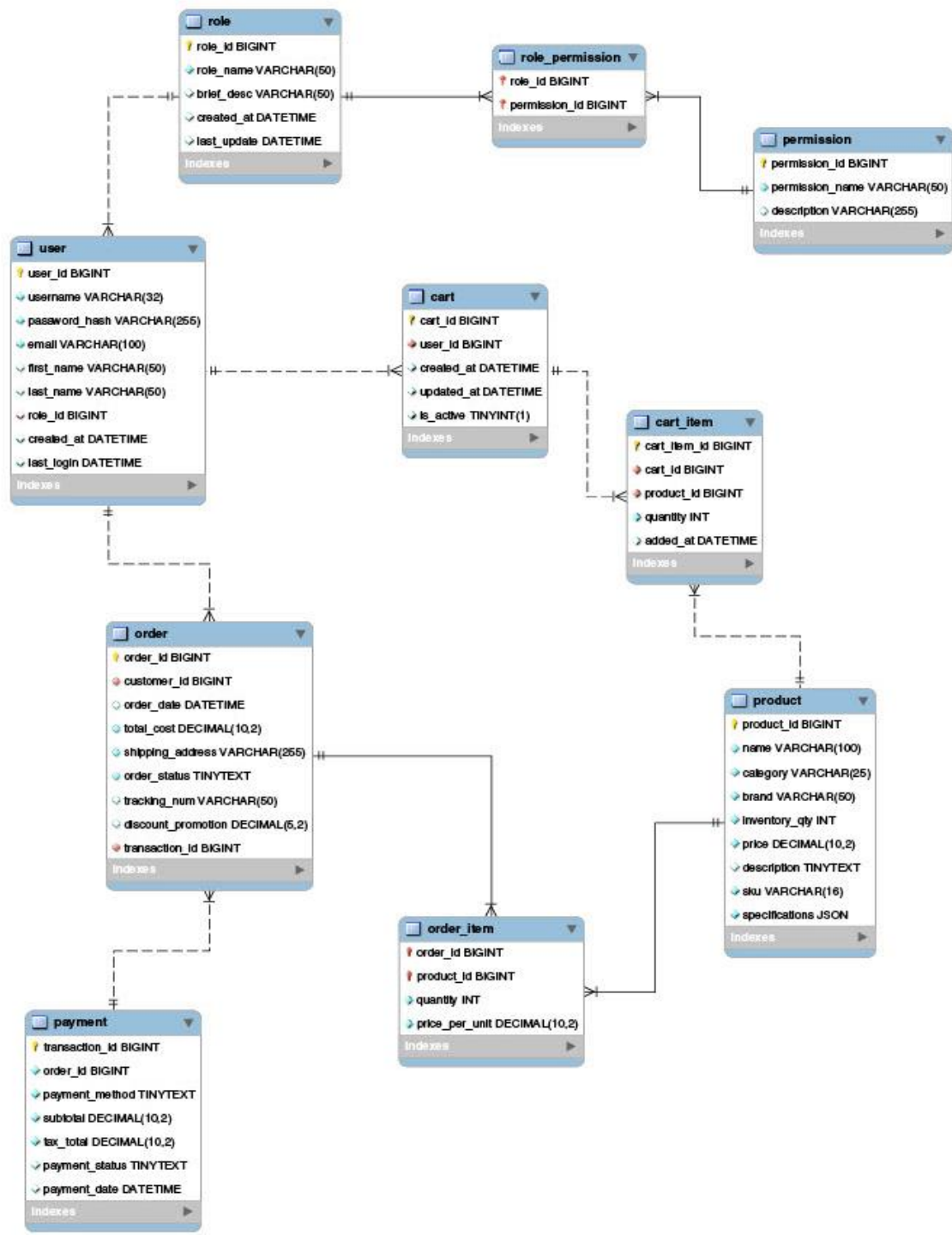
SYSTEM ARCHITECTURE



- Three-tier Architecture:
 - React Frontend
 - Spring Boot Backend
 - MySQL Database
- Modular backend with role enforcement
- Cloud-ready Deployment
- Secure Authentication Flow

Frontend/UI

- React + Vite UI sends information to backend through AXIOS (API Client)
- Uses React Router to enable navigation without requiring a full page reload
 - Manages app routing logic
 - Maps URL's
- Uses React state management to reflect the current state of the UI by handling data that changes overtime
 - Component re-rendering
 - Updates the UI when data changes



Database

- The user, role, permission, and role_permission tables implement a flexible RBAC system to manage user privileges across the platform.
- Users are linked to carts, orders, and payments, allowing a single source of truth for all customer activity and history.
- The product table includes specifications as a JSON field, supporting flexible product attributes like size, specs, or colors.
- Each cart holds multiple cart_item entries tied to product records, while finalized orders are stored in the order and order_item tables, ensuring transactional integrity.

APIs

- APIs are grouped into controllers such as `user-controller` and `products-controller`, offering clean, modular endpoints.
- Endpoints were designed to be stateless using standard HTTP methods and be resource specific.
- Endpoints enable flexible data access and resource targeting using dynamic path parameters.
- Split concerns by endpoint, as well as using authentication context to produce user-specific responses
- ACID compliant endpoints for `/checkout`
- The project includes an OpenAPI 3.1 interface at `/api-docs`, allowing live endpoint testing and schema visibility for developers.

cart-controller

PUT /carts/items

POST /carts/items

GET /carts/me

DELETE /carts

admin-user-controller

PUT /admin/users/{id}/role

GET /admin/users

POST /admin/users

GET /admin/users/{id}

DELETE /admin/users/{id}

admin-product-controller

GET /admin/products/{id}

PUT /admin/products/{id}

DELETE /admin/products/{id}

GET /admin/products

POST /admin/products

checkout-controller

POST /checkout

auth-controller

POST /auth/register

POST /auth/login

Challenges

- Integration of Spring Security with stateless JWT authorization
 - Configuring token lifecycle management
- Knowledge Sharing
 - Most members had limited knowledge base and experience with stack
- Data consistency across different layers and mapping between objects
- Issues with project velocity
 - Issues with uneven contribution
 - Required late-stage realignment of responsibilities

Scalability And Future Work

- Integration with third-party payment providers like Stripe or PayPal
- Fully-featured administrative dashboard for inventory and order processing
- App containerization through Docker, simplifying deployment and scalability
- Enhancing test coverage

End Product

- Modular, full stack e-commerce platform
 - Account registration
 - Authentication
 - Product Browsing
 - Cart Management
- Final architecture supports both scalability and extensibility